

Ejemplos de código — Entrevista React

Preguntas 9, 12, 18, 22–30, 39, 40, 44–45, 49, 52, 55, 61–64, 67–69, 73–78, 82, 84, 97

React base

9. Controlled vs Uncontrolled

EJ. 1 — CONTROLADO

```
function Search() {
  const [q, setQ] = useState('')
  return (
    <input
      value={q}
      onChange={e => setQ(e.target.value)}
      aria-label="Buscar"
    />
  )
}
```

EJ. 2 — NO CONTROLADO + REF

```
function ProfileForm() {
  const nameRef = useRef<HTMLInputElement>(null)
  const submit = () => {
    const name = nameRef.current?.value ?? ''
    saveProfile({ name })
  }
  return <input ref={nameRef} defaultValue="" />
}
```

EJ. 3 — REACT HOOK FORM (HÍBRIDO)

```
const { register, handleSubmit } = useForm<Form>()
return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <input {...register('email')} />
  </form>
)
```

Hooks (18, 22–30)

18. useEffect vs useLayoutEffect

EJ. 1 — USEEFFECT (ANALYTICS, NO BLOQUEA PAINT)

```
useEffect(() => {
  trackPageView(pathname)
}, [pathname])
```

EJ. 2 — USELAYOUTEFFECT (MEDIR DOM, SIN FLICKER)

```
const ref = useRef<HTMLDivElement>(null)
const [h, setH] = useState(0)

useLayoutEffect(() => {
  if (ref.current) setH(ref.current.offsetHeight)
}, [items])

return <div ref={ref} style={{ height: h }}>...</div>
```

23. useCallback

EJ. 1 — HIJO MEMOIZADO

```
const onSelect = useCallback((id: string) => {
  setSelected(id)
}, [])

return <MemoRow item={row} onSelect={onSelect} />
```

EJ. 2 — DEP DE USEEFFECT

```
const load = useCallback(async () => {
  const data = await fetchPolicies(filters)
  setPolicies(data)
}, [filters])

useEffect(() => { load() }, [load])
```

Hooks (18, 22–30)

22. useMemo

EJ. 1 — CÁLCULO CARO

```
const visible = useMemo(
  () => rows.filter(r => r.active && r.score > min),
  [rows, min],
)
```

EJ. 2 — REFERENCIA ESTABLE (COLUMNDEFS)

```
const columnDefs = useMemo(
  () => buildColumns(locale),
  [locale],
)
```

24. useReducer

EJ. 1 — WIZARD MULTI-STEP

```
type State = { step: number; data: FormData }
type Action =
  | { type: 'NEXT' }
  | { type: 'SET_FIELD'; field: string; value: string }

function reducer(s: State, a: Action): State {
  switch (a.type) {
    case 'NEXT': return { ...s, step: s.step + 1 }
    case 'SET_FIELD':
      return { ...s, data: { ...s.data, [a.field]: a.value } }
    default: return s
  }
}

const [state, dispatch] = useReducer(reducer, { step: 0, data: {} })
```

25. Custom hook

EJ. 1 — USEDEBOUNCE

```
function useDebounce<T>(value: T, ms = 300) {
  const [debounced, setDebounced] = useState(value)
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), ms)
    return () => clearTimeout(t)
  }, [value, ms])
  return debounced
}

// const q = useDebounce(search, 300)
```

27. useId

```
function Field({ label }: { label: string }) {
  const id = useId()
  return (
    <>
      <label htmlFor={id}>{label}</label>
      <input id={id} aria-describedby={` ${id}-hint`} />
      <span id={` ${id}-hint`} >Opcional</span>
    </>
  )
}
```

29. Stale closure

EJ. 1 — BUG

```
useEffect(() => {
  const id = setInterval(() => {
    setCount(count + 1) // siempre count inicial
  }, 1000)
  return () => clearInterval(id)
}, []) // ❌ deps vacías
```

EJ. 2 — FIX FUNCIONAL

```
setInterval(() => {
  setCount(c => c + 1) // ✅ siempre actual
}, 1000)
```

26. useContext (split)

EJ. 1 — CONTEXTS SEPARADOS

```
const ThemeCtx = createContext('light')
const UserCtx = createContext<User | null>(null)

// ❌ un solo AppContext con theme+user+cart
// ✅ consumidor de tema no re-render por cambio de cart
```

EJ. 2 — VALOR MEMOIZADO

```
const value = useMemo(
  () => ({ user, signOut }),
  [user, signOut],
)
return <UserCtx.Provider value={value}>{children}</UserCtx.Provider>
```

28. useTransition / useDeferredValue

EJ. 1 — USETRANSITION

```
const [pending, startTransition] = useTransition()
const [filter, setFilter] = useState('')

const onChange = (v: string) => {
  setFilter(v) // input instantáneo
  startTransition(() => setHeavyFilter(v)) // lista diferida
}
```

EJ. 2 — USEDEFERREDVALUE

```
const [text, setText] = useState('')
const deferred = useDeferredValue(text)
const list = useMemo(() => filterHuge(text), [deferred])
```

30. React Compiler

ANTES (MANUAL)

```
const Child = memo(function Child({ onClick }: Props) {
  return <button onClick={onClick}>OK</button>
})
const onClick = useCallback(() => save(), [save])
```

CON COMPILER (CONCEPTO)

```
// El compilador memoiza automáticamente
// si las deps son estables – menos boilerplate
function Child({ onClick }: Props) {
  return <button onClick={onClick}>OK</button>
}
```

Patrones (39–40)

39. Anti-patterns

❌ DERIVAR ESTADO EN USEEFFECT

```
const [fullName, setFullName] = useState('')
useEffect(() => {
  setFullName(`${first} ${last}`) // innecesario
}, [first, last])

// ✅
const fullName = `${first} ${last}`
```

❌ MUTAR STATE

```
items.push(newItem) // ❌
setItems([...items, newItem]) // ✅
```

❌ INDEX COMO KEY

```
{list.map((x, i) => <Row key={i} />)} // ❌
{list.map(x => <Row key={x.id} />)} // ✅
```

40. Suspense

EJ. 1 — LAZY + SUSPENSE

```
const Chart = lazy(() => import('./Chart'))

<Suspense fallback=<ChartSkeleton />>
  <Chart data={data} />
</Suspense>
```

EJ. 2 — NEXT LOADING.TSX

```
// app/dashboard/loading.tsx
export default function Loading() {
  return <DashboardSkeleton />
}

// Suspense boundary automático por segmento
```

EJ. 3 — USE() CON PROMISE (RSC/CLIENT)

```
function Comments({ promise }: { promise: Promise<Comment[]> }) {
  const comments = use(promise)
  return comments.map(c => <li key={c.id}>{c.text}</li>)
}
```

44. Fetch en Server Components**EJ. 1 — CACHE POR DEFECTO (SSG/ISR)**

```
// app/policies/page.tsx — Server Component
export default async function PoliciesPage() {
  const res = await fetch('https://api/policies', {
    next: { revalidate: 60 }, // ISR 60s
  })
  const policies = await res.json()
  return <PolicyList items={policies} />
}
```

EJ. 2 — DINÁMICO SIN CACHE

```
const res = await fetch(url, { cache: 'no-store' })
// o: export const dynamic = 'force-dynamic'
```

45. Route Handlers**EJ. 1 — GET CON ZOD**

```
// app/api/policies/route.ts
import { NextResponse } from 'next/server'
import { parseQuery } from '@schemas/policy'

export async function GET(req: Request) {
  const url = new URL(req.url)
  const parsed = parseQuery(Object.fromEntries(url.searchParams))
  if (!parsed.success)
    return NextResponse.json({ error: parsed.error }, { status: 400 })
  const data = await policyService.list(parsed.data)
  return NextResponse.json(data)
}
```

EJ. 2 — POST

```
export async function POST(req: Request) {
  const body = await req.json()
  const created = await policyService.create(body)
  return NextResponse.json(created, { status: 201 })
}
```

49. Server Actions**EJ. 1 — ACTION + USEACTIONSTATE**

```
// actions.ts
'use server'
export async function updatePolicy(_: State, fd: FormData) {
  const parsed = schema.safeParse({ id: fd.get('id') })
  if (!parsed.success) return { error: 'Invalid' }
  await db.policy.update(parsed.data)
  revalidatePath('/policies')
  return { ok: true }
}

// form.tsx — 'use client'
const [state, action, pending] = useActionState(updatePolicy, {})
```

EJ. 2 — INVOCAR DESDE BOTÓN

```
<form action={deletePolicy}>
  <button type="submit">Delete</button>
</form>
```

52. Parallel & Intercepting Routes**ESTRUCTURA — MODAL CON URL**

```
app/
  layout.tsx           // {children} {modal}
  @modal/
    default.tsx        // null
    (.)photo/[id]/
      page.tsx         // intercepta /photo/[id]
    photo/[id]/page.tsx // página full al refresh directo
```

LAYOUT.TSX

```
export default function Layout({
  children, modal,
}): { children: ReactNode; modal: ReactNode } {
  return (
    <>
      {children}
      {modal}
    </>
  )
}
```

55. Caching Next 15**EJ. 1 — FETCH + TAGS**

```
await fetch(url, { next: { tags: ['policies'] } })

// Tras mutación:
import { revalidateTag } from 'next/cache'
revalidateTag('policies')
```

EJ. 2 — REVALIDATEPATH

```
revalidatePath('/dashboard')
```

EJ. 3 — ROUTER CACHE (CLIENTE)

```
// Navegación back/forward usa cache cliente ~30s
// Forzar fresh: router.refresh()
```

Estado global (61–64)**61. Selectores memoizados (reselect)**

```
const selectActivePolicies = createSelector(
  [(s: RootState) => s.policies.byId,
   (s: RootState) => s.policies.allIds],
  (byId, ids) => ids.map(id => byId[id]).filter(p => p.active),
)

// useSelector(selectActivePolicies)
// Solo recalcula si byId o ids cambian
```

62. Redux vs Context**CONTEXT — TEMA (POCO CAMBIO)**

```
const theme = useContext(ThemeContext)
```

REDUX — UPDATES FRECUENTES + DEVTOOLS

```
const items = useSelector(s => s.cart.items)
dispatch(cartSlice.actions.addItem(product))
```

63. Server vs Client state

SERVER — REACT QUERY

```
const { data } = useQuery({
  queryKey: ['policies', id],
  queryFn: () => fetchPolicy(id),
})
```

CLIENT — UI LOCAL

```
const [modalOpen, setModalOpen] = useState(false)
const [step, setStep] = useState(1)
```

64. Zustand / Jotai

ZUSTAND — STORE SIMPLE

```
const useUIStore = create({ sidebar: boolean;
  toggle: () => void } > (set => ({
    sidebar: false,
    toggle: () => set(s => ({ sidebar: !s.sidebar })),
  })))
const open = useUIStore(s => s.sidebar)
```

JOTAI — ÁTOMO

```
const countAtom = atom(0)
const [count, setCount] = useAtom(countAtom)
```

Performance (67–69, 73–78)

67. React.memo

```
const Row = memo(function Row({ item, onSelect }: Props) {
  return (
    <tr onClick={() => onSelect(item.id)}>
      <td>{item.name}</td>
    </tr>
  )
}, (prev, next) => prev.item.id === next.item.id)
```

68. Virtualización

```
import { useVirtualizer } from '@tanstack/react-virtual'

const parentRef = useRef<HTMLDivElement>(null)
const rowVirtualizer = useVirtualizer({
  count: rows.length,
  getScrollElement: () => parentRef.current,
  estimateSize: () => 36,
})

// Solo renderiza ~20 filas visibles de 100k
```

69. Code splitting

REACT.LAZY

```
const Admin = lazy(() => import('./AdminPanel'))
```

NEXT DYNAMIC

```
const Map = dynamic(() => import('./Map'), {
  ssr: false,
  loading: () => <MapSkeleton />,
})
```

73. Debounce vs Throttle

DEBOUNCE — BÚSQUEDA

```
const debouncedQ = useDebounce(query, 300)
useEffect(() => { searchApi(debouncedQ) }, [debouncedQ])
```

THROTTLE — SCROLL

```
useEffect(() => {
  const onScroll = throttle(() => setY(window.scrollY), 100)
  window.addEventListener('scroll', onScroll)
  return () => window.removeEventListener('scroll', onScroll)
}, [])
```

74. Bundle analysis

```
// next.config.ts
import bundleAnalyzer from '@next/bundle-analyzer'
const withAnalyzer = bundleAnalyzer({ enabled: true })
export default withAnalyzer({ /* config */ })

// ANALYZE=true npm run build
```

75. Tree shaking

✓ NAMED IMPORT ESM

```
import { debounce } from 'lodash-es'
```

✗ IMPORT DEFAULT / BARREL

```
import _ from 'lodash' // trae todo
import { x } from '@/utils' // barrel enorme
```

76. Prefetch / preload

NEXT LINK

```
<Link href="/policies" prefetch>Policies</Link>
```

REACT QUERY HOVER

```
onMouseEnter={() =>
  queryClient.prefetchQuery({
    queryKey: ['policy', id],
    queryFn: () => fetchPolicy(id),
  })
}
```

77. Optimistic updates

REACT QUERY

```
useMutation({
  mutationFn: toggleLike,
  onMutate: async (id) => {
    await queryClient.cancelQueries({ queryKey: ['posts'] })
    const prev = queryClient.getQueryData(['posts'])
    queryClient.setQueryData(['posts'], old =>
      old.map(p => p.id === id ? { ...p, liked: true } : p))
    return { prev }
  },
  onError: (_e, _id, ctx) =>
    queryClient.setQueryData(['posts'], ctx.prev),
})
```

USEOPTIMISTIC (REACT 19)

```
const [optimistic, add] = useOptimistic(items,
  (state, newItem) => [...state, newItem])
```

78. Memoización excesiva (anti)

```
// ✗ memo en cada átomo sin profiler
const Label = memo(({ text }) => <span>{text}</span>)

// ✓ solo hot path medido
const GridCell = memo(HeavyCell) // 10k celdas
```

82. Generics en componentes

```

type ListProps<T> = {
  items: T[]
  keyFn: (item: T) => string
  render: (item: T) => ReactNode
}

function List<T>({ items, keyFn, render }: ListProps<T>) {
  return (
    <ul>
      {items.map(item => (
        <li key={keyFn(item)}>{render(item)}</li>
      ))}
    </ul>
  )
}

// <List items={policies} keyFn={p => p.id} render={p => p.name} />

```

84. Utility types

```

type Policy = { id: string; name: string; premium: number }

type PolicyCard = Pick<Policy, 'id' | 'name'>
type PolicyDraft = Partial<Policy>
type PolicyUpdate = Omit<Policy, 'id'>
type ById = Record<string, Policy>

const routes = {
  home: '/',
  policies: '/policies',
} as const satisfies Record<string, `/${string}`>

```

97. Manejo de errores en producción**EJ. 1 — ERROR.TSX (SEGMENTO NEXT)**

```

'use client'
export default function Error({ error, reset }: {
  error: Error & { digest?: string }
  reset: () => void
}) {
  useEffect(() => { logToSentry(error) }, [error])
  return (
    <main>
      <h1>Algo falló</h1>
      <button onClick={reset}>Reintentar</button>
    </main>
  )
}

```

EJ. 2 — BOUNDARY POR WIDGET + RETRY

```

<ErrorBoundary fallback={<ChartError onRetry={refetch} />}>
  <SalesChart />
</ErrorBoundary>

```

EJ. 3 — API LAYER CON BACKOFF

```

async function fetchWithRetry(url: string, retries = 3) {
  for (let i = 0; i < retries; i++) {
    try {
      const res = await fetch(url)
      if (res.ok) return res.json()
      if (res.status < 500) throw new ApiError(res.status)
    } catch (e) {
      if (i === retries - 1) throw e
      await sleep(2 ** i * 200) // backoff solo 5xx/red
    }
  }
}

```